

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 4

Виконав
студент групи КН-25-1
Гордієнко В. Ю.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Алгоритми пошуку та їх складність

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Порядок виконання роботи

1. Оцінити асимптотичну складність алгоритму лінійного пошуку у O нотації в найгіршому і в найкращому випадку. Як можна покращити алгоритм лінійного пошуку?

1.1. Оцінка асимптотики

- Найгірший випадок : $O(n)$ – шуканий елемент знаходиться в самому кінці списку або взагалі відсутній, тому доводиться перевірити всі n елементів.
- Найкращий випадок : $O(1)$ – елемент знаходиться на першій позиції списку, пошук завершується за 1 ітерацію.

1.2. Як покращити алгоритм

- Метод транспозиції : після кожного успішного знаходження елемент зсувається на одну позицію ближче до початку, що прискорює повторний пошук цього ж елемента.
- Метод «переміщення в початок» : знайдений елемент відразу стає першим.
- Вартовий : зменшує кількість перевірок в циклі (замість перевірки виходу за межі масиву шуканий елемент записується у фіктивний додатковий кінець масиву).

2. Оцінити асимптотичну складність алгоритму бінарного пошуку у O нотації в найгіршому і в найкращому випадку.

2.1. Оцінка асимптотики

- Найгірший випадок : $O(\log_2 n)$ – на кожному кроці простір пошуку скорочується вдвічі, поки не залишиться один елемент.
- Найкращий випадок : $O(1)$ – центральний елемент масиву на першій же ітерації виявляється шуканим.

3. Побудувати алгоритм тернарного пошуку і оцінити його асимптотичну складність алгоритму у O -нотації в найгіршому і в найкращому випадку. Який з алгоритмів є оптимальнішим: бінарний, чи тернарний? Обґрунтувати відповідь відповідними обчисленнями.

3.1. Алгоритм тернарного пошуку

Тернарний пошук ділить масив не на дві, а на три рівні частини за допомогою двох точок розподілу : $mid1 = left + (right - left) / 3$ та $mid2 = right - (right - left) / 3$.

3.2. Оцінка асимптотики

- Складність у найгіршому випадку : $O(\log_3 n)$
- Складність у найкращому випадку : $O(1)$

3.3. Який з алгоритмів оптимальніший

Бінарний пошук є оптимальнішим на практиці. Попри те, що функція $\log_3 n$ зростає повільніше за $\log_2 n$, кількість порівнянь на кожній ітерації у тернарному пошуку більша (мінімум 2-4 порівняння замість 1-2 у бінарному). Загальний час роботи тернарного пошуку через додаткові розгалуження більший приблизно на 50%, ніж у бінарного.

4. Порівняти ефективність алгоритмів лінійного, бінарного та тернарного пошуку для різних розмірів вхідного списку. Для цього провести 29 експериментальне дослідження та побудувати графіки залежності часу виконання алгоритму від розміру вхідного списку.

4.1. Порівняння ефективності алгоритмів

- Лінійний пошук демонструє лінійне зростання часу. Для великих масивів стає вкрай неефективним.
- Бінарний пошук демонструє логарифмічне зростання. Навіть для мільйонних списків виконується майже миттєво (до 20 операцій).
- Тернарний пошук також має логарифмічну криву, але графік лежить трохи вище графіка бінарного пошуку через константні накладні витрати на обчислення двох серединних точок.

5. Порівняти алгоритми пошуку за їхньою здатністю працювати з відсортованими та не відсортованими списками. Провести аналіз впливу відсортованості списку на час виконання кожного алгоритму.

5.1. Порівняння та аналіз впливу відсортованості списку на алгоритми

Лінійний пошук: працює як з відсортованими, так і з невідсортованими списками. Відсортованість не впливає на складність найгіршого випадку $O(n)$, проте у відсортованому масиві лінійний пошук можна зупинити раніше, якщо поточний елемент став більшим за шуканий.

Бінарний та Тернарний пошук: працюють виключно з відсортованими списками. Якщо список не відсортований, ці алгоритми дадуть некоректний результат, оскільки порушується логіка відсікання сегментів. Час попереднього сортування (наприклад, $O(n \log n)$) нівелює вигоду, якщо пошук виконується одноразово.

6. Розглянути сценарії використання кожного з алгоритмів пошуку у практичних задачах і обґрунтувати вибір кожного алгоритму в конкретному випадку.

6.1. Сценарії практичного використання

- Лінійний пошук: Маленькі масиви (до 10-20 елементів), динамічні структури даних, де елементи постійно додаються в довільному порядку, або пошук у незв'язних списках.
- Бінарний пошук: Стабільні великі бази даних, словники, файлові системи, де дані відсортовані заздалегідь або сортуються один раз, а пошук виконується багаторазово.
- Тернарний пошук: Використовується рідко в класичних масивах через програш бінарному. Проте його концепція незамінна для пошуку екстремумів (мінімумів/максимумів) унімодальних функцій, де аналітичний розрахунок похідної неможливий.

Висновки: на цій практичній роботі я опанував основні алгоритми сортування та навчився методам аналізу їх асимптотичної складності.

Контрольні питання

1. Що таке алгоритм пошуку і чому він важливий у контексті комп'ютерних наук?

Алгоритм пошуку – це покрокова процедура для знаходження конкретного елемента (або інформації) у структурі даних. Він є фундаментальним, оскільки пошук є однією з найчастіших операцій у будь-якому ПЗ (бази даних, пошукові рушії, операційні системи), і від його швидкості залежить загальна продуктивність систем.

2. Які основні критерії оцінки ефективності алгоритмів пошуку?

Головними критеріями є:

- Часова складність (кількість операцій/порівнянь у найкращому, середньому та найгіршому випадках);
- Просторова складність (кількість додаткової пам'яті, що потрібна для роботи алгоритму).

3. Що таке лінійний пошук, і як він працює?

Лінійний пошук – це найпростіший алгоритм, який послідовно перебирає кожен елемент структури даних (від першого до останнього) і порівнює його з шуканим значенням, допоки не знайде збіг або не закінчиться масив.

4. Які умови повинні бути виконані для успішного застосування бінарного пошуку?

Необхідно виконання двох умов:

- Дані у масиві повинні бути строго відсортовані за зростанням або спаданням.
- Має бути забезпечено прямий доступ до елементів за індексом за сталий час $O(1)$ (наприклад, масив, а не зв'язний список).

5. Які переваги та недоліки використання бінарного пошуку порівняно з іншими алгоритмами?

- Переваги: дуже висока швидкість роботи на великих об'ємах даних ($O(\log n)$), мінімальні витрати пам'яті $O(1)$.

- Недоліки: вимагає обов'язкової відсортованості даних; неефективний на динамічних структурах, де дані часто змінюються, оскільки підтримка відсортованості коштує дорого.